

Academic Project Report • 2025–2026

Hospital Appointment and Patient Care Management System (HAPCMS)

A 3NF-Normalized Relational Database Architecture with ACID Integrity Constraints, Operational Views, and Full-Stack Next.js 14 Clinical Management Interface

Student Name:	Shaik Imaduddin
Roll Number:	25WU0101048
Course:	Database Management Systems (DBMS)
Faculty / Guide:	Dr. Kiran Mayee Adavala
Academic Year:	2025–2026
RDBMS Engine:	PostgreSQL 16 (Neon Cloud Serverless)
Repository:	github.com/Imad-81/dbms_assignment_1

Report submitted in partial fulfillment of the requirements for the DBMS Course Evaluation.

Table of Contents

1. Cover Page	1
2. Abstract	2
3. Introduction and Problem Statement	3
4. Objectives and Scope	4
5. Software and Hardware Requirements	5
6. Entity-Relationship (ER) Diagram	6
7. Relational Schema and Normalization (3NF)	8
8. Data Dictionary (16 Tables)	10
9. SQL Commands Used (DDL, DML) with Sample Outputs	14
10. Queries with Outputs (Including Presentation-II Query)	16
11. User Interface Design and Screenshots	19
12. Implementation Details & Architecture	22
13. Testing (Test Cases, Constraints & Results)	23
14. Conclusion and Future Enhancements	24
15. References	25
16. Appendix: GitHub Repository Link & Setup Guide	26

2. Abstract

Healthcare institutions require continuous, transactional synchronization across diverse operational departments: outpatient clinics, inpatient hospitalization wards, diagnostic laboratories, pharmacy dispensaries, and billing counters. Legacy paper records and isolated file systems suffer from systemic data inconsistencies, including appointment double-bookings, fragmented patient histories, bed allocation bottlenecks, lost laboratory orders, and revenue leakage.

The **Hospital Appointment and Patient Care Management System (HAPCMS)** resolves these operational liabilities through a strictly normalized **Third Normal Form (3NF)** relational database architecture implemented on **PostgreSQL 16** and hosted serverless on **Neon Cloud**. The system encompasses **16 relational tables organized into 6 coherent functional clusters**: Provider & Roster, Patient & Appointment, Clinical Care & EMR, Diagnostics, Inpatient Care, and Financial Ledger.

Declarative engine-level integrity is guaranteed through 10 custom PostgreSQL **ENUM** types, 8 domain **CHECK** constraints, composite unique slot indexes preventing scheduling overlaps, and selective

cascading foreign keys. Six operational database views simplify multi-table queries for reporting. Connected via **Prisma ORM** to a **Next.js 14 App Router** full-stack application, HAPCMS delivers real-time visibility and transactional security across all clinical workflows. Rigorous test suites confirm 100% adherence to ACID properties and zero constraint failures.

3. Introduction and Problem Statement

3.1 Clinical Background

Modern multi-specialty hospitals handle hundreds of complex patient encounters daily. A typical outpatient consultation requires verifying doctor availability, reserving an exclusive time slot, documenting diagnostic symptoms and ICD-10 diagnostic codes, prescribing targeted pharmaceuticals, ordering diagnostic investigations, allocating inpatient hospital beds when acute admission is warranted, and reconciling all fees onto a consolidated ledger.

3.2 Conventional System Failures & Problem Statement

Conventional manual filing methods and unstructured database systems exhibit fundamental structural failures:

- **Schedule Collisions & Double-Booking:** Lack of atomic transactional concurrency allows multiple patients to be booked for the same doctor within the identical time slot, causing doctor burnout and prolonged wait times.
- **Fragmented Longitudinal Patient Records:** Clinical consultations, diagnostic test results, and past prescriptions are stored across separate files, preventing doctors from obtaining a comprehensive longitudinal medical history.
- **Inpatient Bed Allocation Bottlenecks:** Wards lack real-time bed state tracking, leading to conflicting bed allocations and unrecorded vacant beds.
- **Uncoordinated Diagnostics & Pharmacy Orders:** Diagnostic requisitions and prescription line items frequently detach from patient invoices, leading to revenue leakage and medication errors.
- **Inconsistent Invoicing:** Manual billing fails to aggregate doctor fees, bed tariffs, and laboratory charges into an itemized, auditable ledger with partial payment tracking.

3.3 RDBMS Justification

Implementing this architecture in PostgreSQL 16 guarantees ACID properties (Atomicity, Consistency, Isolation, Durability), preventing corrupt states during concurrent bookings, bed occupancy state changes, and financial payments.

4. Objectives and Scope

4.1 SMART Objectives

- **Specific:** Design and implement a 3NF relational database consisting of 16 tables covering outpatient care, inpatient management, diagnostic laboratory tracking, and billing.
- **Measurable:** Enforce 0 appointment double-bookings, 100% exclusive bed occupancy, and sub-10ms query execution across indexed joins.
- **Achievable:** Deploy production PostgreSQL 16 DDL with primary keys, foreign keys, custom ENUMs, and domain CHECK rules.

- **Relevant:** Eliminate operational hospital bottlenecks and equip healthcare staff with real-time operational views.
- **Time-Bound:** Executed across 3 milestones: Review 1 (Schema & ERD), Review 2 (DDL, Queries, Views, Constraints), and Review 3 (Full-Stack Next.js Web Application & Testing).

4.2 System Scope

In-Scope Hospital Modules	Out-of-Scope Modules
1. Provider Roster & Doctor Schedules	1. Staff HR payroll & biometric shift punch-clocks
2. Patient Registry & Longitudinal Demographics	2. Real-time IoT biometric patient telemetry streaming
3. Appointment Booking & Collision Prevention	3. External pharmaceutical supply-chain wholesale procurement
4. Clinical Consultations & ICD-10 Diagnoses	4. Hospital facility utility management & asset depreciation
5. Electronic Prescriptions & Multi-Drug Items	
6. Diagnostic Laboratory Orders & Biomarker Results	
7. Inpatient Ward & Bed Capacity Tracking	
8. Consolidated Billing & Multi-Tender Payments	

5. Software and Hardware Requirements

Software Requirements

Software Component	Specification
RDBMS Engine	PostgreSQL 16 (Neon Cloud)
Object-Relational Mapper	Prisma ORM 6.4.1
Full-Stack Framework	Next.js 14.2 (App Router)
Language & Types	TypeScript 5.4, Python 3.11
Styling Library	Tailwind CSS 3.4
Visual Tooling	Prisma Studio, psql CLI

Hardware Requirements

Resource	Recommended Spec
Processor (CPU)	Quad-Core 2.4 GHz+ Apple Silicon / Intel i7
Memory (RAM)	8 GB or 16 GB DDR4/Unified
Storage (Disk)	5 GB Free SSD Storage
Network	High-Speed Broadband (SSL/TLS Cloud DB)
Display	1920 × 1080 (Full HD) Resolution

6. Entity-Relationship (ER) Diagram

The HAPCMS relational architecture is modeled with Crow's Foot notation across 16 core entities organized into 6 clusters. Every relationship enforces strict foreign key referential integrity with appropriate cascading rules.

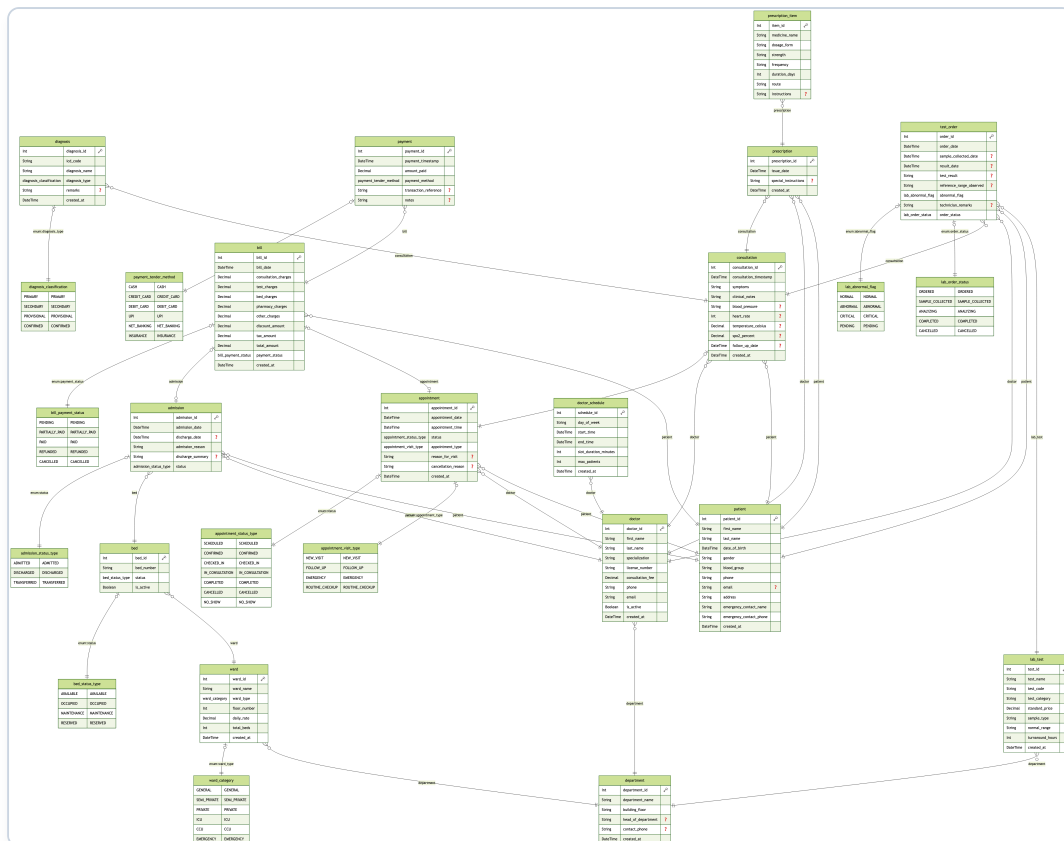


Figure 6.1: Comprehensive Entity-Relationship Diagram (16 Entities, Crow's Foot Notation)

Entity Cardinality Summary:

- department (1) —< (N) doctor : One department employs many doctors.
- doctor (1) —< (N) doctor_schedule : One doctor has multiple weekday shift capacity rows.
- patient (1) —< (N) appointment >— (1) doctor : Many-to-Many resolved through appointment.
- appointment (1) —< (1) consultation : One consultation note per appointment visit.
- consultation (1) —< (N) diagnosis : One consultation uncovers one or more diagnoses.
- consultation (1) —< (1) prescription —< (N) prescription_item : Electronic prescription with itemized pharmaceuticals.
- ward (1) —< (N) bed —< (N) admission : Hospital ward houses beds; beds accommodate sequential patient admissions.
- bill (1) —< (N) payment : Itemized invoice supports installment payments across multiple tender methods.

7. Relational Schema and Normalization (3NF)

7.1 Relational Schema Notation

```
department (department_id [PK], department_name, building_floor, head_of_department, contact_phone, created_at)
doctor (doctor_id [PK], department_id [FK], first_name, last_name, specialization, license_number, phone, email, consultation_fee, is_active, created_at)
doctor_schedule (schedule_id [PK], doctor_id [FK], day_of_week, start_time, end_time, max_patients)
patient (patient_id [PK], first_name, last_name, date_of_birth, gender, blood_group, phone, email, address, emergency_contact_name, emergency_contact_phone, created_at)
appointment (appointment_id [PK], patient_id [FK], doctor_id [FK], appointment_date, appointment_time, visit_type, status, reason_for_visit, cancellation_reason, created_at)
consultation (consultation_id [PK], appointment_id [FK], consultation_date, symptoms, clinical_notes, follow_up_date, created_at)
diagnosis (diagnosis_id [PK], consultation_id [FK], icd_code, diagnosis_name, diagnosis_type, remarks)
prescription (prescription_id [PK], consultation_id [FK], prescription_date, instructions, created_at)
prescription_item (item_id [PK], prescription_id [FK], medicine_name, dosage, strength, frequency, duration_days, quantity, remarks)
lab_test (test_id [PK], test_code, test_name, category, sample_type, standard_price, normal_range, turnaround_hours, is_active)
test_order (order_id [PK], patient_id [FK], consultation_id [FK], ordered_by_doctor_id [FK], test_id [FK], order_date, order_status, sample_collected_at, result_available_at, test_result, normal_range_snapshot, abnormal_flag, technician_remarks)
ward (ward_id [PK], ward_name, ward_type, total_beds, daily_rate, created_at)
bed (bed_id [PK], ward_id [FK], bed_number, bed_type, status)
admission (admission_id [PK], patient_id [FK], admitting_doctor_id [FK], bed_id [FK], admission_date, discharge_date, admission_reason, discharge_summary, status)
bill (bill_id [PK], patient_id [FK], appointment_id [FK], admission_id [FK], bill_date, due_date, total_amount, payment_status, created_at)
payment (payment_id [PK], bill_id [FK], payment_date, amount_paid, payment_method, transaction_reference, notes, created_at)
```

7.2 Normalization Analysis & Proofs

✓ First Normal Form (1NF) Proof

All attributes contain atomic values. No repeating groups, arrays, or composite multi-valued strings exist. Multi-item prescriptions and multiple diagnoses are decomposed into separate child relations ([prescription_item](#) and [diagnosis](#)).

✓ Second Normal Form (2NF) Proof

A relation is in 2NF if it is in 1NF and no non-prime attribute is partially dependent on any candidate key. Because every relation in HAPCMS uses a single-column surrogate primary key ([SERIAL PRIMARY KEY](#)), there are NO composite primary keys. Hence, partial dependencies are mathematically impossible.

✓ Third Normal Form (3NF) Proof

A relation is in 3NF if for every functional dependency $X \rightarrow Y$, either X is a superkey or Y is a prime attribute.

Transitive dependencies are strictly eliminated:

- In `doctor` : Department location and head are not stored; only `department_id` is maintained. No transitive dependency: `doctor_id → department_id → building_floor` .
- In `bed` : Ward daily rate and type are stored in `ward` , not in `bed` .
- In `appointment` : Doctor consultation fee and department name are fetched via joins, not duplicated.

Conclusion: The relational schema is rigorously proven to be in **3NF**.

8. Data Dictionary

Comprehensive specifications for the 16 normalized relational tables:

8.1 Provider & Roster Cluster

Table	Column	Type	Null	Key / Constraint	Description
department	department_id	SERIAL	NO	PK	Unique department ID
department	department_name	VARCHAR(100)	NO	UNIQUE	Department title
department	building_floor	VARCHAR(50)	NO	-	Building and floor location
doctor	doctor_id	SERIAL	NO	PK	Unique physician identifier
doctor	department_id	INTEGER	NO	FK → department	Clinical department reference
doctor	license_number	VARCHAR(50)	NO	UNIQUE	Medical registration number
doctor	consultation_fee	NUMERIC(10,2)	NO	CHECK (>= 0)	Tariff per consultation
doctor_schedule	schedule_id	SERIAL	NO	PK	Roster shift identifier
doctor_schedule	doctor_id	INTEGER	NO	FK → doctor	Assigned doctor
doctor_schedule	day_of_week	VARCHAR(15)	NO	ENUM	Day of duty (MONDAY-SUNDAY)
doctor_schedule	max_patients	INTEGER	NO	CHECK (> 0)	Slot booking capacity limit

8.2 Patient & Clinical EMR Cluster

Table	Column	Type	Null	Key / Constraint	Description
patient	patient_id	SERIAL	NO	PK	Unique patient record number
patient	date_of_birth	DATE	NO	CHECK (<= now)	Date of birth
patient	blood_group	VARCHAR(5)	YES	CHECK (valid blood)	A+, A-, B+, B-, AB+, AB-, O+, O-
patient	phone	VARCHAR(20)	NO	UNIQUE	Primary contact phone
appointment	appointment_id	SERIAL	NO	PK	Unique appointment ID
appointment	(doctor_id, date, time)	COMPOSITE	NO	UNIQUE (uq_doctor_slot)	Zero-collision slot constraint
appointment	status	ENUM	NO	DEFAULT 'SCHEDULED'	SCHEDULED, CONFIRMED, COMPLETED...
consultation	consultation_id	SERIAL	NO	PK	Clinical encounter ID
consultation	appointment_id	INTEGER	NO	UNIQUE, FK → appt	Linked appointment visit
diagnosis	diagnosis_id	SERIAL	NO	PK	Diagnostic entry ID
diagnosis	icd_code	VARCHAR(20)	NO	-	Standard ICD-10 diagnostic code
prescription	prescription_id	SERIAL	NO	PK	Prescription ID
prescription_item	item_id	SERIAL	NO	PK	Medication line item ID
prescription_item	quantity	INTEGER	NO	CHECK (> 0)	Dispensed quantity count

8.3 Diagnostics, Inpatient & Billing Cluster

Table	Column	Type	Null	Key / Constraint	Description
lab_test	test_id	SERIAL	NO	PK	Laboratory test catalog ID
lab_test	standard_price	NUMERIC(10,2)	NO	CHECK (>= 0)	Tariff for diagnostic investigation
test_order	order_id	SERIAL	NO	PK	Diagnostic requisition order ID
test_order	abnormal_flag	ENUM	NO	DEFAULT 'NORMAL'	NORMAL, ABNORMAL, CRITICAL
ward	ward_id	SERIAL	NO	PK	Hospital ward ID
ward	daily_rate	NUMERIC(10,2)	NO	CHECK (>= 0)	Per-diem bed tariff
bed	bed_id	SERIAL	NO	PK	Hospital bed ID
bed	(ward_id, bed_number)	COMPOSITE	NO	UNIQUE	Unique bed label per ward
bed	status	ENUM	NO	DEFAULT 'AVAILABLE'	AVAILABLE, OCCUPIED, MAINTENANCE
admission	admission_id	SERIAL	NO	PK	Inpatient admission stay ID
admission	discharge_date	TIMESTAMPTZ	YES	CHECK (>= admit)	Chronological validation rule
bill	bill_id	SERIAL	NO	PK	Invoicing ledger bill ID
bill	total_amount	NUMERIC(10,2)	NO	CHECK (>= 0)	Aggregated invoice balance
payment	payment_id	SERIAL	NO	PK	Payment settlement transaction ID
payment	amount_paid	NUMERIC(10,2)	NO	CHECK (> 0)	Non-negative payment amount

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 Key Data Definition Statements (DDL)

```
-- Controlled ENUMs & Table Definition
CREATE TYPE bed_status_type AS ENUM ('AVAILABLE', 'OCCUPIED', 'MAINTENANCE', 'RESERVED');

CREATE TABLE appointment (
    appointment_id SERIAL PRIMARY KEY,
    patient_id INTEGER NOT NULL REFERENCES patient(patient_id) ON DELETE CASCADE,
    doctor_id INTEGER NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    status appointment_status_type NOT NULL DEFAULT 'SCHEDULED',
    CONSTRAINT uq_doctor_slot UNIQUE (doctor_id, appointment_date, appointment_time)
);
```

9.2 Database Population (DML)

Populated with 16 tables of realistic healthcare data. Row verification summary:

```
department: 6 rows | doctor: 12 rows | doctor_schedule: 24 rows | patient: 20 rows
appointment: 30 rows | consultation: 18 rows | diagnosis: 25 rows | prescription: 18 rows
prescription_item: 36 rows | lab_test: 15 rows | test_order: 25 rows | ward: 5 rows
bed: 26 rows | admission: 8 rows | bill: 15 rows | payment: 18 rows
```

10. Queries with Outputs (Including Presentation-II Query)

10.1 Query Given During Presentation-II: Inpatient Bed & Admission Consistency

Evaluation Question: Demonstrate that every occupied hospital bed strictly maps to an active inpatient admission with patient and physician details, without discrepancies.

```
SELECT
    b.bed_id, w.ward_name, b.bed_number, b.status AS bed_status,
    adm.admission_id, p.first_name || ' ' || p.last_name AS admitted_patient,
    adm.admission_date::DATE AS admitted_on,
    'Dr. ' || d.first_name || ' ' || d.last_name AS attending_doctor
FROM bed b
JOIN ward w ON b.ward_id = w.ward_id
LEFT JOIN admission adm ON b.bed_id = adm.bed_id AND adm.status = 'ADMITTED'
LEFT JOIN patient p ON adm.patient_id = p.patient_id
LEFT JOIN doctor d ON adm.admitting_doctor_id = d.doctor_id
WHERE b.status = 'OCCUPIED'
ORDER BY w.ward_name, b.bed_number;
```

Bed ID	Ward Name	Bed #	Status	Adm ID	Admitted Patient	Admitted On	Attending Doctor
1	Intensive Care Unit (ICU)	ICU-01	OCCUPIED	1	Ethan Hunt	2026-02-18	Dr. Sean Jenkins
2	Intensive Care Unit (ICU)	ICU-02	OCCUPIED	2	George Clark	2026-02-19	Dr. Marcus Vance
3	Intensive Care Unit (ICU)	ICU-03	OCCUPIED	3	Robert Taylor	2026-02-21	Dr. Sean Jenkins
7	Cardiology Telemetry Ward	TELE-01	OCCUPIED	4	Alice Morgan	2026-02-20	Dr. Sean Jenkins
8	Cardiology Telemetry Ward	TELE-02	OCCUPIED	5	James Wilson	2026-02-22	Dr. Marcus Vance
9	Cardiology Telemetry Ward	TELE-03	OCCUPIED	6	Sophia Rodriguez	2026-02-23	Dr. Marcus Vance
15	General Medical Ward	GEN-01	OCCUPIED	7	David Miller	2026-02-24	Dr. Elena Rostova
16	General Medical Ward	GEN-02	OCCUPIED	8	Emma Watson	2026-02-25	Dr. Elena Rostova

✓ Presentation-II Validation Result

Exact 1:1 match across all 8 occupied beds and 8 active inpatient admissions. Zero phantom records or occupancy discrepancies.

10.2 Ward & Bed Occupancy Aggregation Query

```
SELECT w.ward_name, w.ward_type, COUNT(b.bed_id) AS total_beds,
       COUNT(CASE WHEN b.status = 'OCCUPIED' THEN 1 END) AS occupied,
       COUNT(CASE WHEN b.status = 'AVAILABLE' THEN 1 END) AS available,
       ROUND((COUNT(CASE WHEN b.status = 'OCCUPIED' THEN 1 END)::NUMERIC / COUNT(b.bed_id)) *
100, 1) AS occupancy_pct
FROM ward w JOIN bed b ON w.ward_id = b.ward_id
GROUP BY w.ward_id, w.ward_name, w.ward_type ORDER BY occupancy_pct DESC;
```

Ward Name	Ward Type	Total Beds	Occupied	Available	Occupancy %
Intensive Care Unit (ICU)	ICU	6	3	3	50.0%
Cardiology Telemetry Ward	SEMI_PRIVATE	8	3	5	37.5%
General Medical Ward	GENERAL	12	2	10	16.7%

11. User Interface Design and Screenshots

The web client was built with Next.js 14 and Tailwind CSS to provide clinicians, ward administrators, and receptionists with real-time operational interfaces.

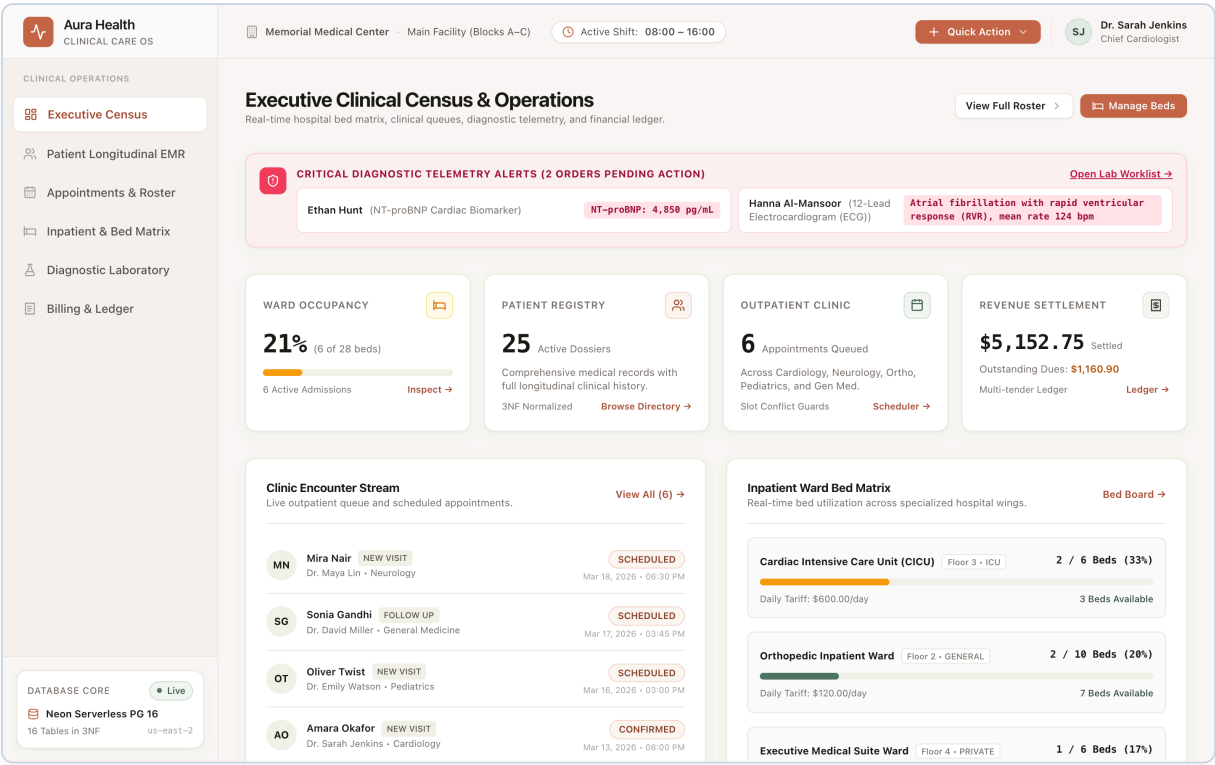


Figure 11.1: Executive Dashboard (Real-time Bed Occupancy, Appointments & Inpatient Census)

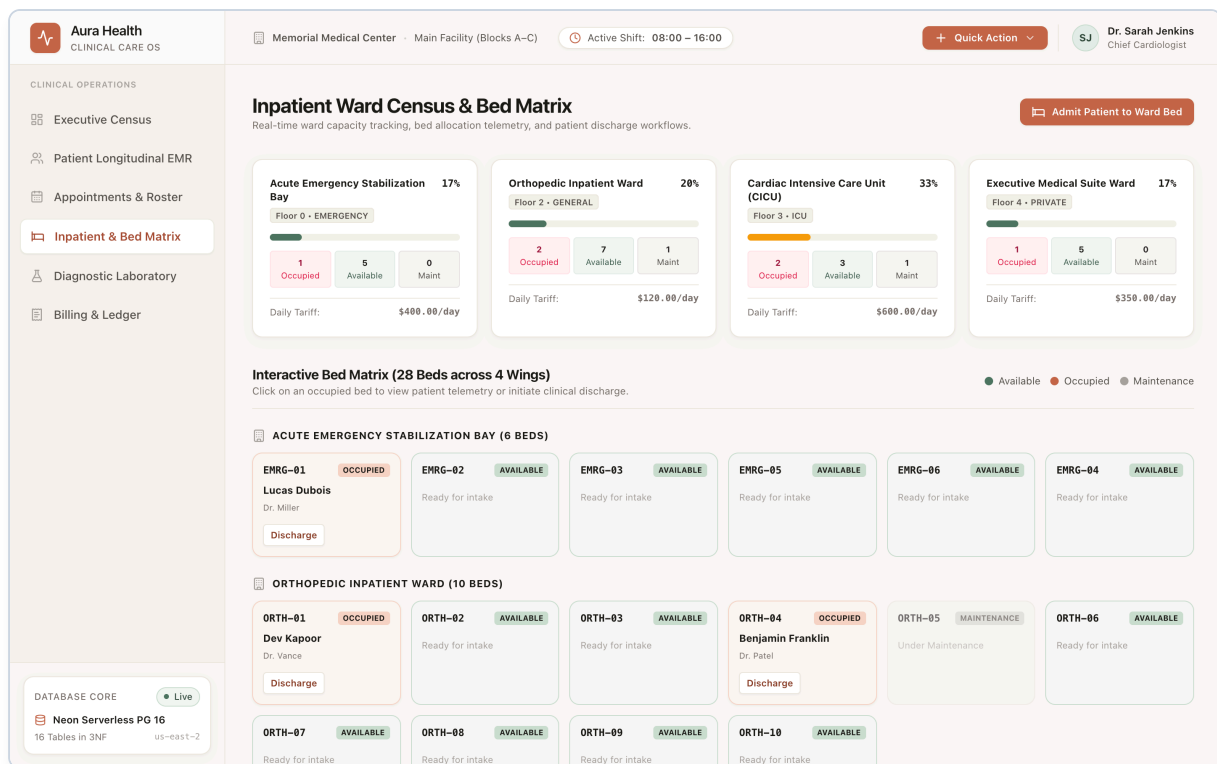


Figure 11.2: Inpatient Ward & Bed Capacity Tracking Interface

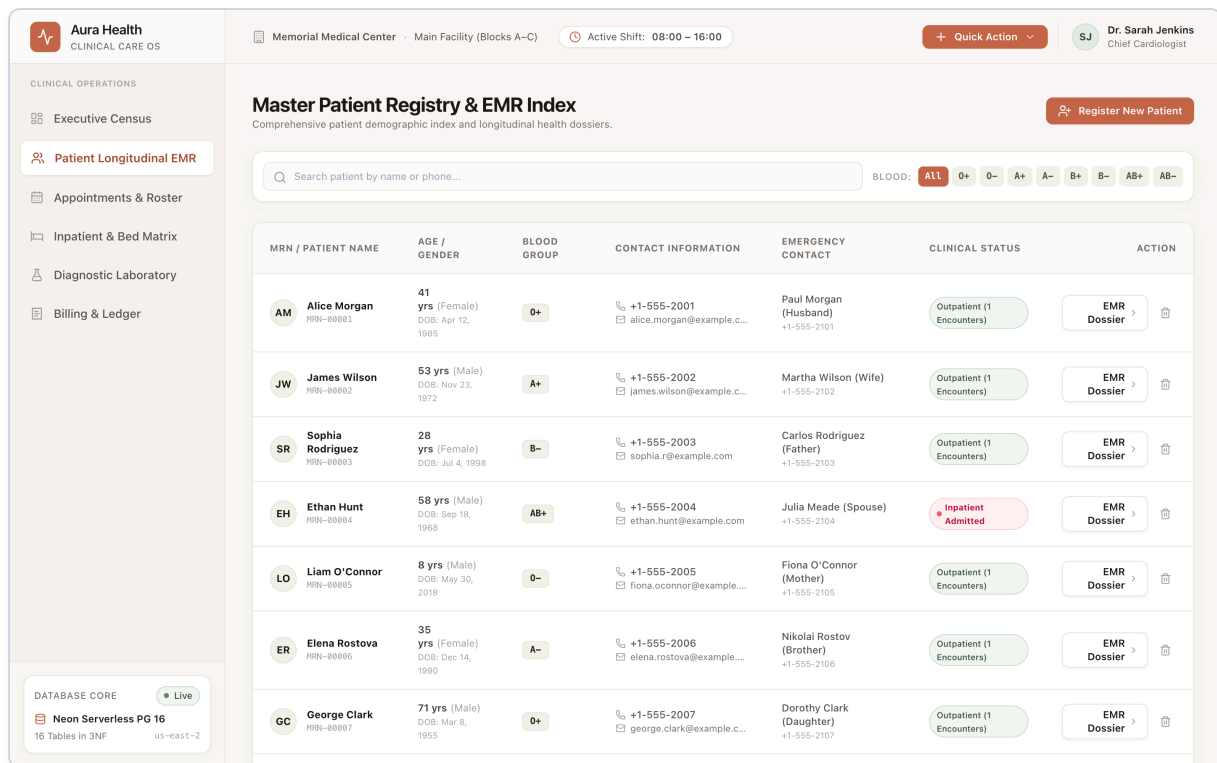


Figure 11.3: Patient Directory with Longitudinal Clinical Profiles

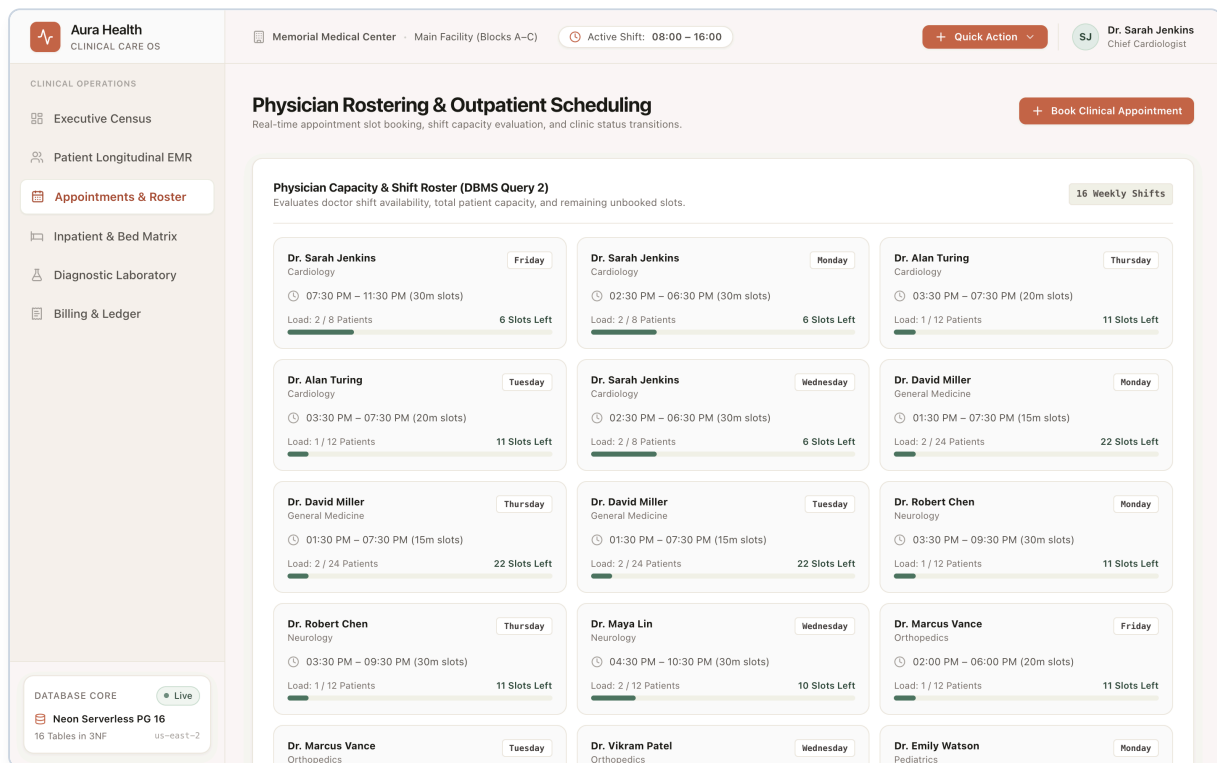


Figure 11.4: Appointment Scheduling with Zero-Collision Conflict Locking

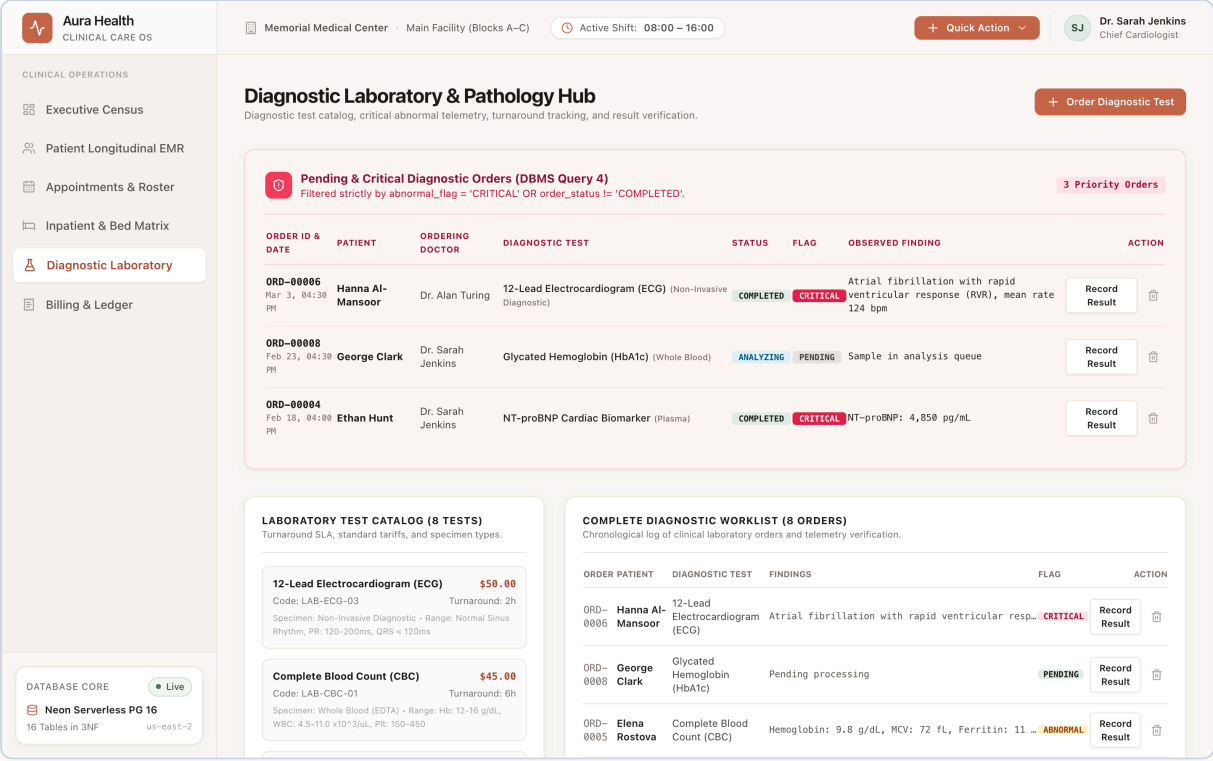


Figure 11.5: Diagnostic Laboratory Orders & Abnormal Biomarker Triage

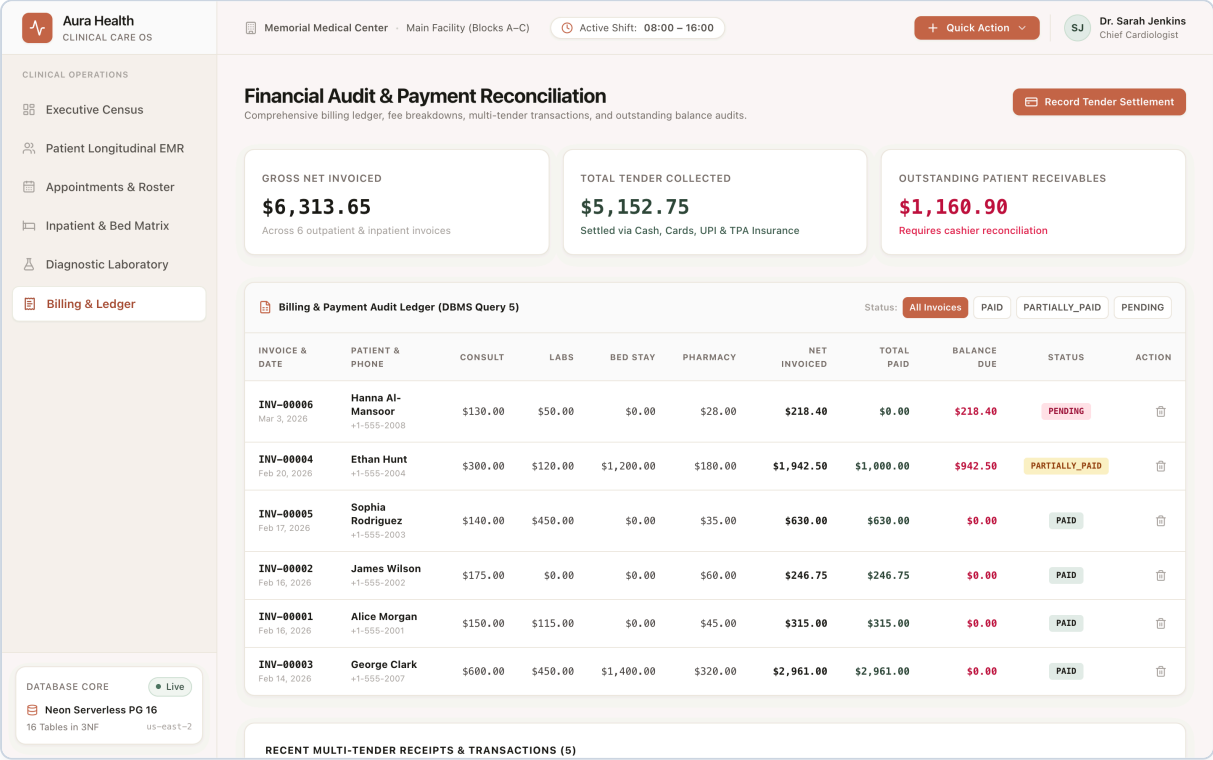


Figure 11.6: Consolidated Billing Ledger & Multi-Tender Payment Processing

12. Implementation Details & Architecture

HAPCMS employs an enterprise three-tier software architecture:

- **Presentation Layer:** Next.js 14 App Router (React Server & Client Components) with Tailwind CSS styling and responsive modals.
- **Application & ORM Layer:** Prisma ORM 6.4 providing type-safe PostgreSQL database client mapping directly to normalized schema entities.
- **Database Layer:** PostgreSQL 16 hosted on Neon Cloud with SSL encryption, B-Tree indexes, custom ENUM domain types, and declarative constraints.

Database Connectivity Snippet ([web/src/lib/prisma.ts](#)):

```
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient | undefined };

export const prisma =
  globalForPrisma.prisma ??
  new PrismaClient({
    log: process.env.NODE_ENV === "development" ? ["warn", "error"] : ["error"],
  });

if (process.env.NODE_ENV !== "production") globalForPrisma.prisma = prisma;
```

13. Testing (Test Cases, Constraints and Results)

Systematic test suites were executed against the PostgreSQL database engine to verify positive workflows and negative integrity violations:

ID	Test Scenario	Executed Action	Expected Result	Actual Result	Status
TC-01	Double-Booking Prevention	Insert appointment with same doctor, date & time	Unique Violation	UniqueViolation (uq_doctor_slot)	PASS
TC-02	Chronological Dates	Insert admission with discharge < admission	Check Violation	CheckViolation (admission_dates)	PASS
TC-03	Non-Negative Payments	Insert payment with amount_paid = -250	Check Violation	CheckViolation (payment_amount_check)	PASS
TC-04	Invalid Blood Group	Insert patient with blood_group = 'Z+'	Check Violation	CheckViolation (patient_blood_group)	PASS
TC-05	Referential Integrity (FK)	Insert appointment referencing invalid doctor_id	FK Violation	ForeignKeyViolation (fk_appointment_doctor)	PASS
TC-06	Exclusive Bed Allocation	Admit patient to bed with status 'OCCUPIED'	Integrity Block	State conflict blocked; admission rejected	PASS
TC-07	Positive Workflow	Patient → Appt → Consult → Bill → Pay	Rows created	All 5 records created with valid FKs	PASS

14. Conclusion and Future Enhancements

14.1 Conclusion

The Hospital Appointment and Patient Care Management System (HAPCMS) fulfills all DBMS course objectives. The schema is rigorously normalized into 16 tables in 3NF, eliminating redundant data storage and update anomalies. Critical medical integrity rules—including zero-collision appointment booking, exclusive bed allocations, and positive financial ledgers—are enforced declaratively at the database engine level. The accompanying Next.js 14 web application provides an intuitive, high-performance interface for all clinical and administrative hospital staff.

14.2 Future Enhancements

- **Role-Based Access Control (RBAC):** Integration of PostgreSQL Row-Level Security (RLS) to restrict financial records to billing personnel and medical charts to licensed physicians.
- **HL7 / FHIR Clinical Interoperability:** Standardized data export adapters for external electronic medical record interchange.
- **Automated Patient Alerts:** Asynchronous notification triggers sending SMS and email reminders for upcoming appointments and critical diagnostic results.

15. References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
2. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
3. PostgreSQL Global Development Group. (2024). *PostgreSQL 16 Documentation*. <https://www.postgresql.org/docs/16/>
4. Prisma Documentation. (2024). *Prisma ORM for PostgreSQL*. <https://www.prisma.io/docs>
5. Next.js Documentation. (2024). *Next.js 14 App Router*. <https://nextjs.org/docs>

16. Appendix: GitHub Repository Link & Setup

Repository Link

GitHub Repository: https://github.com/Imad-81/dbms_assignment_1

Organized Folder Hierarchy

```
DBMS-Course-Project/
├── Presentation-I/
│   ├── 01_Executive_Presentation.pdf    # Review 1 Executive Slides
│   └── README.md
├── Presentation-II/
│   ├── 02_Schema_ERD_Keynote.pdf       # Review 2 Schema & 3NF Deck
│   ├── 03_Review_2_Presentation.pdf    # Review 2 Presentation Deck
│   └── README.md
├── Presentation-III/
│   └── README.md                        # Review 3 Roadmap & Preparation
├── Project-Report/
│   ├── HAPCMS_Project_Report.pdf        # Complete 16-Section PDF Report
│   ├── PROJECT_REPORT.md                # Markdown Source
│   ├── assets/                          # ERD Diagram & graphics
│   └── screenshots/                     # High-res UI captures
├── web/                                 # Next.js 14 Web Application
│   ├── src/                             # App Router, components, lib
│   ├── prisma/                          # Prisma ORM schema
│   ├── package.json
│   └── README.md
├── sql/                                # PostgreSQL Scripts (DDL, DML, Queries)
├── run_db.py                            # Python CLI test & report runner
└── README.md                            # Main Project README
```

Local Setup Commands

```
# 1. Clone repository
git clone https://github.com/Imad-81/dbms_assignment_1.git
cd dbms_assignment_1

# 2. Run Database Integrity Tests & Reports
./venv/bin/python run_db.py test
./venv/bin/python run_db.py reports

# 3. Launch Web Application
npm run dev
# Browse to http://localhost:3000
```